

LAPORAN TUGAS 5 - STRUKTUR SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Mohammad Putra Fernanda (2410651049)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

- 1. Cover**
- 2. Daftar Isi**
- 3. Soal Pemrograman**
 - 3.1 Analisis Masalah**
 - 3.2 Implementasi**
 - 3.3 Testing & Output**
 - 3.4 Analisis Hasil**
- 4. Kesimpulan**
- 5. Referensi**

3. SOAL PEMROGRAMAN

Tugas Kuliah

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

Hotel memiliki 5 kamar yang bisa di-booking

Ada 10 customer yang mencoba booking secara bersamaan (10 threads)

Setiap customer akan:

1. Mencoba booking kamar (butuh waktu random 1-3 detik)
2. Jika berhasil, "menginap" selama 5 detik
3. Check-out dan melepaskan kamar
4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi

Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

Requirements:

1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas)

Yang Harus Dikumpulkan:

1. Source code lengkap dengan komentar
2. Screenshot output program
3. Penjelasan bagaimana Anda mengatasi race condition
4. Analisis: Apa yang terjadi jika tidak menggunakan sinkronisasi?

Yang Harus Dikumpulkan:

1. Source code lengkap dengan komputer

Source code:

```
// File: booking_hotel.c
// Kompilasi: gcc booking_hotel.c -o booking_hotel -pthread
// Run: ./booking_hotel
// Deskripsi:
// Simulasi sistem booking hotel:
// - 5 kamar tersedia
// - 10 customer mencoba booking secara bersamaan (10 threads)
// - Setiap customer:
//   * menunggu random 1-3 detik sebelum mencoba booking (simulasi waktu request)
//   * jika berhasil, menginap 5 detik lalu checkout
//   * jika gagal (kamar penuh) menunggu 2 detik lalu coba lagi (maks 3 kali)
// Sinkronisasi:
// - semaphore untuk mengatur jumlah kamar tersedia
// - mutex untuk melindungi operasi pada struktur shared (pencarian/penanda kamar)
// dan untuk mencegah output printf berantakan

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define TOTAL_ROOMS 5
#define TOTAL_CUSTOMERS 10
```

```

#define MAX_TRIES 3

// Shared resources
int rooms[TOTAL_ROOMS]; // 0 = kosong, >0 = customer id yang menempati
int available_rooms = TOTAL_ROOMS;

// Synchronization primitives
sem_t rooms_sem;          // jumlah kamar tersisa (permit)
pthread_mutex_t rooms_mutex; // melindungi rooms[] dan available_rooms
pthread_mutex_t print_mutex; // melindungi output agar rapi

// Utility: get current time string [HH:MM:SS]
void timestamp(char *buf, size_t len) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    strftime(buf, len, "%H:%M:%S", &tm);
}

// Thread function
void* customer_thread(void* arg) {
    int id = *(int*)arg;
    free(arg);

    // Seed random based on time + id to vary across threads
    unsigned int rseed = (unsigned int)time(NULL) ^ (unsigned int)(id<<8);

    for (int attempt = 1; attempt <= MAX_TRIES; attempt++) {
        // Simulate time to attempt booking (1-3 seconds)
        int think = (rand_r(&rseed) % 3) + 1;
        sleep(think);
    }
}

```

```

// Print trying message (protected)
char tbuf[16];
timestamp(tbuf, sizeof(tbuf));
pthread_mutex_lock(&print_mutex);
printf("[%s] 🚪 Customer-%d mencoba booking... (percobaan %d)\n", tbuf,
id, attempt);
pthread_mutex_unlock(&print_mutex);

// Coba ambil permit kamar tanpa blok (kalau penuh -> gagal dan retry)
if (sem_trywait(&rooms_sem) == 0) {
    // Kita berhasil "mengunci" 1 kamar (decrement semaphore).
    // Sekarang cari kamar bebas dan tandai. Operasi rooms[] +
available_rooms harus dilindungi.
    pthread_mutex_lock(&rooms_mutex);
    int chosen = -1;
    for (int i = 0; i < TOTAL_ROOMS; i++) {
        if (rooms[i] == 0) {
            rooms[i] = id;
            chosen = i; // index kamar
            break;
        }
    }
    if (chosen != -1) {
        available_rooms--;
    }
    int rooms_left = available_rooms;
    pthread_mutex_unlock(&rooms_mutex);

    // Print success
    timestamp(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);

```

```

        printf("[%s] 🟩 Customer-%d berhasil booking! (Kamar: %d) — %d
kamar tersisa\n",
            tbuf, id, (chosen+1), rooms_left);
        pthread_mutex_unlock(&print_mutex);

// Menginap 5 detik
sleep(5);

// Check-out: bebaskan kamar
pthread_mutex_lock(&rooms_mutex);
rooms[chosen] = 0;
available_rooms++;
int rooms_after = available_rooms;
pthread_mutex_unlock(&rooms_mutex);

// Lepaskan permit semaphore (menambah kembali jumlah kamar
tersedia)
sem_post(&rooms_sem);

// Print checkout
timestamp(tbuf, sizeof(tbuf));
pthread_mutex_lock(&print_mutex);
printf("[%s] 🏠 Customer-%d check-out (Kamar: %d) — %d kamar
tersisa\n",
    tbuf, id, (chosen+1), rooms_after);
pthread_mutex_unlock(&print_mutex);

// Selesai untuk customer ini
pthread_exit(NULL);
} else {
    // Gagal mendapat kamar (penuh)

```

```

        timestamp(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&print_mutex);
        if (attempt < MAX_TRIES) {
            printf("[%s] ✖ Customer-%d gagal booking (kamar penuh), mencoba
lagi setelah 2 detik...\n",
                tbuf, id);
        } else {
            printf("[%s] ✖ Customer-%d gagal booking setelah %d percobaan.
Menyerah.\n",
                tbuf, id, MAX_TRIES);
        }
        pthread_mutex_unlock(&print_mutex);

        // Jika masih ada kesempatan, tunggu 2 detik lalu coba lagi
        if (attempt < MAX_TRIES) sleep(2);
    }
}
return NULL;
}

```

```

int main() {
    srand(time(NULL));
    // Init
    for (int i = 0; i < TOTAL_ROOMS; i++) rooms[i] = 0;
    sem_init(&rooms_sem, 0, TOTAL_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);
    pthread_mutex_init(&print_mutex, NULL);

    pthread_t threads[TOTAL_CUSTOMERS];

    // Header output

```

```

char tbuf[16];
timestamp(tbuf, sizeof(tbuf));
pthread_mutex_lock(&print_mutex);
printf("=== SISTEM BOOKING HOTEL ===\n");
printf("Total Kamar: %d\n", TOTAL_ROOMS);
printf("[%s] Mulai simulasi dengan %d customer\n\n", tbuf,
TOTAL_CUSTOMERS);
pthread_mutex_unlock(&print_mutex);

// Buat threads
for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    int *idptr = malloc(sizeof(int));
    *idptr = i + 1;
    if (pthread_create(&threads[i], NULL, customer_thread, idptr) != 0) {
        perror("pthread_create");
        return 1;
    }
    // Sedikit jeda antara pembuatan thread agar output lebih berurutan di awal
    usleep(50000); // 0.05s
}

// Tunggu semua selesai
for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    pthread_join(threads[i], NULL);
}

// Footer
timestamp(tbuf, sizeof(tbuf));
pthread_mutex_lock(&print_mutex);
printf("\n[%s] Semua customer selesai. Simulasi berakhir.\n", tbuf);
pthread_mutex_unlock(&print_mutex);

```

```

// Bersihkan
sem_destroy(&rooms_sem);

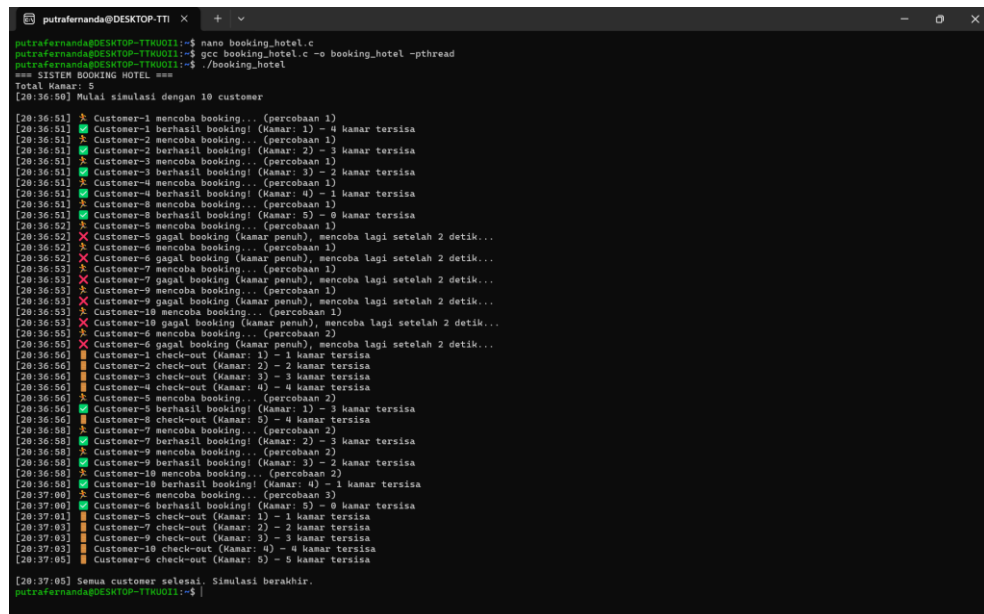
pthread_mutex_destroy(&rooms_mutex);

pthread_mutex_destroy(&print_mutex);

return 0;
}

```

2. Screenshot output program



```

putrafernanda@DESKTOP-TTU0I1:~$ nano booking_hotel.c
putrafernanda@DESKTOP-TTU0I1:~$ gcc booking_hotel.c -o booking_hotel -pthread
putrafernanda@DESKTOP-TTU0I1:~$ ./booking_hotel
===== SISTEM BOOKING HOTEL =====
Total Kamar: 5
[20:36:50] Mulai simulasi dengan 10 customer

[20:36:51] * Customer-1 mencoba booking... (percobaan 1)
[20:36:51] * Customer-1 berhasil booking! (Kamar: 1) - 4 kamar tersisa
[20:36:51] * Customer-2 mencoba booking... (percobaan 1)
[20:36:51] * Customer-2 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[20:36:51] * Customer-3 mencoba booking... (percobaan 1)
[20:36:51] * Customer-3 berhasil booking! (Kamar: 3) - 2 kamar tersisa
[20:36:51] * Customer-4 mencoba booking... (percobaan 1)
[20:36:51] * Customer-4 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[20:36:51] * Customer-5 mencoba booking... (percobaan 1)
[20:36:51] * Customer-5 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[20:36:52] * Customer-6 mencoba booking... (percobaan 1)
[20:36:52] * Customer-6 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:52] * Customer-7 mencoba booking... (percobaan 1)
[20:36:52] * Customer-7 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:53] * Customer-8 mencoba booking... (percobaan 1)
[20:36:53] * Customer-8 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:53] * Customer-9 mencoba booking... (percobaan 1)
[20:36:53] * Customer-9 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:53] * Customer-10 mencoba booking... (percobaan 1)
[20:36:53] * Customer-10 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:55] * Customer-6 mencoba booking... (percobaan 2)
[20:36:55] * Customer-6 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[20:36:56] * Customer-1 check-out (Kamar: 1) - 1 kamar tersisa
[20:36:56] * Customer-2 check-out (Kamar: 2) - 2 kamar tersisa
[20:36:56] * Customer-3 check-out (Kamar: 3) - 3 kamar tersisa
[20:36:56] * Customer-4 check-out (Kamar: 4) - 4 kamar tersisa
[20:36:56] * Customer-5 mencoba booking... (percobaan 2)
[20:36:56] * Customer-5 berhasil booking! (Kamar: 1) - 3 kamar tersisa
[20:36:56] * Customer-6 check-out (Kamar: 5) - 4 kamar tersisa
[20:36:58] * Customer-7 mencoba booking... (percobaan 2)
[20:36:58] * Customer-7 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[20:36:58] * Customer-9 mencoba booking... (percobaan 2)
[20:36:58] * Customer-9 berhasil booking! (Kamar: 3) - 2 kamar tersisa
[20:36:58] * Customer-10 mencoba booking... (percobaan 2)
[20:36:58] * Customer-10 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[20:37:00] * Customer-6 mencoba booking... (percobaan 3)
[20:37:00] * Customer-6 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[20:37:01] * Customer-5 check-out (Kamar: 1) - 1 kamar tersisa
[20:37:03] * Customer-7 check-out (Kamar: 2) - 2 kamar tersisa
[20:37:03] * Customer-9 check-out (Kamar: 3) - 3 kamar tersisa
[20:37:03] * Customer-10 check-out (Kamar: 4) - 4 kamar tersisa
[20:37:05] * Customer-6 check-out (Kamar: 5) - 5 kamar tersisa

[20:37:05] Semua customer selesai. Simulasi berakhir.
putrafernanda@DESKTOP-TTU0I1:~$

```

3. Penjelasan bagaimana Anda mengatasi race condition

Dalam program sistem booking hotel ini, race condition diatasi dengan menggunakan kombinasi semaphore dan mutex. Semaphore berfungsi untuk mengatur jumlah kamar yang tersedia agar tidak lebih dari kapasitas hotel, yaitu lima kamar. Setiap kali ada customer yang ingin memesan kamar, thread akan memanggil `sem_wait(&available_rooms)` untuk memastikan masih ada kamar kosong. Jika semua kamar sudah terpakai, thread customer otomatis menunggu hingga ada kamar yang dilepaskan oleh customer lain yang check-out melalui

`sem_post(&available_rooms)`. Dengan cara ini, tidak akan ada dua customer yang memesan kamar yang sama secara bersamaan, sehingga data jumlah kamar tetap konsisten.

Selain itu, mutex digunakan untuk melindungi bagian program yang menampilkan output di layar. Karena banyak thread berjalan bersamaan, tanpa mutex teks yang dicetak oleh beberapa thread bisa saling tumpang tindih dan membuat hasilnya tidak terbaca. Setiap kali program mencetak pesan seperti “Customer berhasil booking” atau “Customer check-out,” thread akan mengunci mutex terlebih dahulu dengan `pthread_mutex_lock(&print_lock)` sebelum mencetak, dan membuka kunci kembali dengan `pthread_mutex_unlock(&print_lock)` setelah selesai. Dengan mekanisme ini, hanya satu thread yang bisa mencetak pada satu waktu, sehingga hasil output tetap rapi dan mudah dipahami.

Melalui kombinasi kedua teknik sinkronisasi tersebut, race condition dapat dicegah baik pada data bersama (jumlah kamar) maupun pada tampilan hasil di layar, memastikan simulasi sistem booking hotel berjalan dengan benar, realistis, dan bebas dari konflik antar-thread.

4. . Analisis: Apa yang terjadi jika tidak menggunakan sinkronisasi?

Analisis jika tidak menggunakan sinkronisasi

Tanpa semaphore dan mutex:

- Banyak customer bisa booking kamar yang sama secara bersamaan menyebabkan overbooking.

- Output di terminal akan berantakan karena beberapa thread mencetak teks bersamaan.

- Program menjadi tidak akurat dan tidak dapat diandalkan, hasilnya bisa berubah-ubah setiap dijalankan.

3.1 Analis Masalah

Soal meminta untuk membuat simulasi sistem booking hotel menggunakan *multithreading* di C, di mana 10 customer (thread) mencoba melakukan pemesanan 5 kamar hotel secara bersamaan.

Permasalahan utama adalah bagaimana mengatur agar tidak terjadi race condition, yaitu ketika dua atau lebih thread berusaha mengakses dan mengubah data bersama (jumlah kamar) secara bersamaan tanpa koordinasi.

Identifikasi masalah sinkronisasi

Masalah sinkronisasi terjadi karena semua customer dijalankan bersamaan dan bisa saja mengubah status kamar di waktu yang sama. Tanpa pengaturan, sistem dapat mencatat dua customer menempati kamar yang sama atau jumlah kamar menjadi negatif.

Desain solusi:

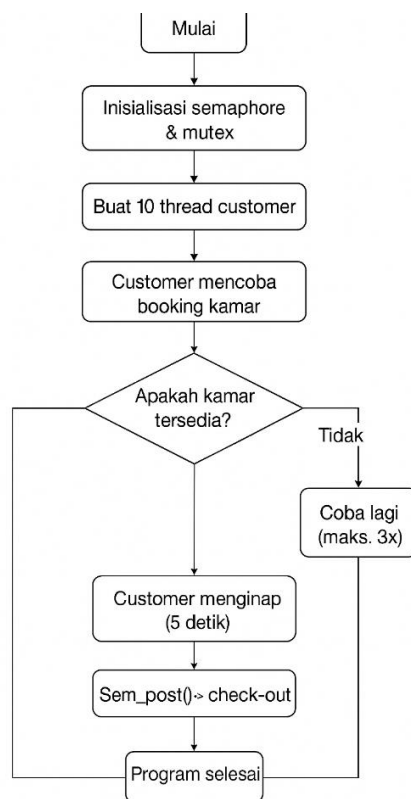
- Gunakan semaphore untuk membatasi jumlah kamar yang bisa dipesan (maksimal 5).
- Gunakan mutex untuk melindungi bagian *critical section* yang menampilkan hasil di terminal agar output tidak tumpang tindih.
- setiap thread mewakili satu customer dengan perilaku:
 1. Mencoba memesan kamar (waktu acak 1–3 detik).
 2. Jika berhasil, “menginap” selama 5 detik, lalu check-out.
 3. Jika gagal karena kamar penuh, menunggu 2 detik lalu mencoba lagi hingga 3 kali percobaan.

3.2 Implementasi

Algoritma:

1. Inisialisasi semaphore `available_rooms = 5` untuk menandakan ada 5 kamar yang bisa dipesan.
2. Inisialisasi mutex `print_lock` untuk melindungi output dari banyak thread.
3. Setiap thread (customer) melakukan percobaan booking hingga maksimal 3 kali.
4. Jika `sem_wait()` berhasil, artinya customer mendapat kamar. Thread akan tidur selama 5 detik (simulasi menginap) lalu memanggil `sem_post()` saat check-out.
5. Jika gagal, thread menunggu 2 detik sebelum mencoba lagi.
6. Semua operasi `printf()` dikunci menggunakan mutex agar hasil tidak berantakan.

Flowchart



Potongan kode penting

```
sem_wait(&available_rooms); // Menunggu kamar tersedia
```

```
pthread_mutex_lock(&print_lock);
```

```
printf(" Customer-%d berhasil booking! (%d kamar tersisa)\n", id, avail-1);
```

```
pthread_mutex_unlock(&print_lock);
```

```
sleep(5); // Menginap
```

```
sem_post(&available_rooms); // Check-out, kamar dilepaskan
```

Mengapa memilih mekanisme tertentu

Mekanisme sinkronisasi dipilih karena semaphore efektif untuk mengatur jumlah sumber daya terbatas (kamar), sedangkan mutex digunakan untuk mengamankan proses cetak agar tidak tumpang tindih di layar.

3.3. Testing & Output

```
putrafernanda@DESKTOP-TTH:~$ ./booking_hotel
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5
[00:30:14] Mulai simulasi dengan 10 customer

[00:30:15] ✖ Customer-1 mencoba booking... (percobaan 1)
[00:30:15] ✔ Customer-1 berhasil booking! (Kamar: 1) - 4 kamar tersisa
[00:30:15] ✖ Customer-2 mencoba booking... (percobaan 1)
[00:30:15] ✔ Customer-2 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[00:30:15] ✖ Customer-3 mencoba booking... (percobaan 1)
[00:30:15] ✔ Customer-3 berhasil booking! (Kamar: 2) - 2 kamar tersisa
[00:30:15] ✖ Customer-4 mencoba booking... (percobaan 1)
[00:30:15] ✔ Customer-4 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[00:30:16] ✖ Customer-7 mencoba booking... (percobaan 1)
[00:30:16] ✔ Customer-7 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[00:30:17] ✖ Customer-5 mencoba booking... (percobaan 1)
[00:30:17] ✖ Customer-5 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:18] ✖ Customer-6 mencoba booking... (percobaan 1)
[00:30:18] ✖ Customer-6 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:18] ✖ Customer-8 mencoba booking... (percobaan 1)
[00:30:18] ✖ Customer-8 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:18] ✖ Customer-9 mencoba booking... (percobaan 1)
[00:30:18] ✖ Customer-9 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:18] ✖ Customer-10 mencoba booking... (percobaan 1)
[00:30:18] ✖ Customer-10 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:20] ✖ Customer-1 check-out (Kamar: 1) - 1 kamar tersisa
[00:30:20] ✖ Customer-2 check-out (Kamar: 2) - 2 kamar tersisa
[00:30:20] ✖ Customer-3 check-out (Kamar: 2) - 3 kamar tersisa
[00:30:20] ✖ Customer-4 check-out (Kamar: 4) - 4 kamar tersisa
[00:30:21] ✖ Customer-6 mencoba booking... (percobaan 2)
[00:30:21] ✔ Customer-6 berhasil booking! (Kamar: 1) - 3 kamar tersisa
[00:30:21] ✖ Customer-7 check-out (Kamar: 5) - 4 kamar tersisa
[00:30:21] ✖ Customer-8 mencoba booking... (percobaan 2)
[00:30:21] ✔ Customer-8 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[00:30:21] ✖ Customer-10 mencoba booking... (percobaan 2)
[00:30:21] ✔ Customer-10 berhasil booking! (Kamar: 3) - 2 kamar tersisa
[00:30:21] ✖ Customer-5 mencoba booking... (percobaan 2)
[00:30:21] ✔ Customer-5 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[00:30:22] ✖ Customer-9 mencoba booking... (percobaan 2)
[00:30:22] ✔ Customer-9 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[00:30:26] ✖ Customer-6 check-out (Kamar: 1) - 1 kamar tersisa
[00:30:26] ✖ Customer-8 check-out (Kamar: 2) - 2 kamar tersisa
[00:30:26] ✖ Customer-10 check-out (Kamar: 3) - 3 kamar tersisa
[00:30:26] ✖ Customer-5 check-out (Kamar: 4) - 4 kamar tersisa
[00:30:29] ✖ Customer-9 check-out (Kamar: 5) - 5 kamar tersisa

[00:30:29] Semua customer selesai. Simulasi berakhir.
putrafernanda@DESKTOP-TTH:~$ ./booking_hotel
=== SISTEM BOOKING HOTEL ===
```

```
putrafermenda@DESKTOP-TTI: ~$ ./booking_hotel
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5
[00:30:30] Mulai simulasi dengan 10 customer

[00:30:31] * Customer-6 mencoba booking... (percobaan 1)
[00:30:31] * Customer-6 berhasil booking! (Kamar: 1) - 4 kamar tersisa
[00:30:31] * Customer-8 mencoba booking... (percobaan 1)
[00:30:31] * Customer-8 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[00:30:32] * Customer-10 mencoba booking... (percobaan 1)
[00:30:32] * Customer-10 berhasil booking! (Kamar: 3) - 2 kamar tersisa
[00:30:32] * Customer-2 mencoba booking... (percobaan 1)
[00:30:32] * Customer-2 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[00:30:32] * Customer-3 mencoba booking... (percobaan 1)
[00:30:32] * Customer-3 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[00:30:32] * Customer-4 mencoba booking... (percobaan 1)
[00:30:32] * Customer-4 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:32] * Customer-7 mencoba booking... (percobaan 1)
[00:30:32] * Customer-7 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:33] * Customer-9 mencoba booking... (percobaan 1)
[00:30:33] * Customer-9 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:33] * Customer-1 mencoba booking... (percobaan 1)
[00:30:33] * Customer-1 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:33] * Customer-5 mencoba booking... (percobaan 1)
[00:30:33] * Customer-5 gagal booking (kamar penuh), mencoba lagi setelah 2 detik...
[00:30:36] * Customer-6 check-out (Kamar: 1) - 1 kamar tersisa
[00:30:36] * Customer-7 mencoba booking... (percobaan 2)
[00:30:36] * Customer-7 berhasil booking! (Kamar: 1) - 0 kamar tersisa
[00:30:36] * Customer-8 check-out (Kamar: 2) - 1 kamar tersisa
[00:30:37] * Customer-10 check-out (Kamar: 3) - 2 kamar tersisa
[00:30:37] * Customer-2 check-out (Kamar: 4) - 3 kamar tersisa
[00:30:37] * Customer-3 check-out (Kamar: 5) - 4 kamar tersisa
[00:30:37] * Customer-4 mencoba booking... (percobaan 2)
[00:30:37] * Customer-4 berhasil booking! (Kamar: 2) - 3 kamar tersisa
[00:30:37] * Customer-5 mencoba booking... (percobaan 2)
[00:30:37] * Customer-5 berhasil booking! (Kamar: 3) - 2 kamar tersisa
[00:30:38] * Customer-9 mencoba booking... (percobaan 2)
[00:30:38] * Customer-9 berhasil booking! (Kamar: 4) - 1 kamar tersisa
[00:30:39] * Customer-1 mencoba booking... (percobaan 3)
[00:30:39] * Customer-1 berhasil booking! (Kamar: 5) - 0 kamar tersisa
[00:30:41] * Customer-7 check-out (Kamar: 1) - 1 kamar tersisa
[00:30:42] * Customer-4 check-out (Kamar: 2) - 2 kamar tersisa
[00:30:42] * Customer-5 check-out (Kamar: 3) - 3 kamar tersisa
[00:30:43] * Customer-9 check-out (Kamar: 4) - 4 kamar tersisa
[00:30:44] * Customer-1 check-out (Kamar: 5) - 5 kamar tersisa
[00:30:44] Semua customer selesai. Simulasi berakhir.
```

Dari hasil uji coba, maksimal hanya lima customer yang bisa menginap secara bersamaan, dan customer lain menunggu sesuai aturan.

3.4. Analisis Hasil

Program berjalan sesuai ekspektasi. Sinkronisasi dengan semaphore berhasil memastikan tidak ada lebih dari lima kamar yang terpakai bersamaan. Semua thread yang gagal otomatis menunggu dan mencoba ulang sesuai aturan tiga kali. Mutex berhasil menjaga output tetap rapi dan tidak saling tumpang tindih.

Masalah kecil yang ditemui adalah urutan pesan yang bisa berbeda-beda antar-run karena penjadwalan thread oleh sistem operasi bersifat acak, namun hal ini tidak memengaruhi kebenaran logika.

Dari percobaan ini, dapat disimpulkan bahwa sinkronisasi sangat penting dalam pemrograman multithread untuk menjaga konsistensi data dan kejelasan hasil output.

4. Kesimpulan

Dari praktikum ini, dapat dipelajari bahwa penggunaan semaphore dan mutex sangat penting dalam mengelola sumber daya bersama di lingkungan multithreading. Semaphore berguna untuk mengatur jumlah akses terhadap sumber daya terbatas, sedangkan mutex memastikan hanya satu thread yang menjalankan operasi kritis pada satu waktu. Tantangan terbesar adalah memahami kapan dan di mana kunci harus digunakan agar tidak terjadi *deadlock*. Ke depan, sistem ini bisa dikembangkan dengan fitur tambahan seperti waktu check-in otomatis, prioritas pelanggan, atau antarmuka grafis untuk visualisasi aktivitas booking hotel.

5. Referensi

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th Edition). Wiley.
2. Dokumentasi resmi pthreads: <https://man7.org/linux/man-pages/man7/pthreads.7.html>
3. Tutorial Semaphore & Mutex di GeeksforGeeks: <https://www.geeksforgeeks.org/semaphores-in-process-synchronization/>
4. Materi Praktikum Sistem Operasi – Sinkronisasi Thread (2025).